SatsPath Security Audit Report

Target: SatsPath Protocol v1 Prototype (Resolution + Routing + Experimental Swaps) **Scope:** White-box code review (Rust crates + TypeScript Ark Bridge) + Black-box protocol analysis **Date:** 2026-06-30 **Classification:** Hackathon Prototype — Do Not Use With Real Funds

---

Executive Summary

SatsPath is a well-designed resolution and routing layer with strong cryptographic foundations. The codebase demonstrates **excellent security hygiene** for a prototype: mandatory signature verification, fail-closed defaults, private material detection, encrypted swap storage, and comprehensive ownership proof infrastructure.

However, as a **prototype**, it has known gaps documented in `threat_model.md` and several **critical findings** that must be addressed before any mainnet deployment.

Critical Findings (Must Fix)

1. CRITICAL: Fee Estimation Trust Violation — Single Point of Failure

Location: `crates/satspath-router/src/fees.rs:28-45` **Impact:** Routing decisions for on-chain vs Ark are based on `mempool.space` API. If attacker controls DNS or MITM, they can force all large payments to Ark (or on-chain at high fees).

```
// fees.rs - Single external API, no redundancy, no signature verification
async fn fetch_fee_estimate() -> FeeEstimate {
    match try_fetch_fee_estimate().await {
        Ok(est) => est,
        Err(_) => FeeEstimate::fallback(),  // Silent fallback to 10 sat/vB
    }
}
```

Attack Vectors: - DNS hijacking → route all payments to Ark server (censorship) - MITM → inflate `fastest_fee` to 500 sat/vB → force Ark fallback - API compromise → arbitrary fee manipulation

Mitigation Required: - Multiple fee sources (mempool.space + blockstream.info + local node) - Consensus: reject if sources disagree by > 2x - Optional: pinned TLS certs for fee API endpoints - User-configurable fee source with local Bitcoin Core RPC as ground truth

---

2. CRITICAL: BIP-353 DNSSEC Fails Closed but Has No Production Validator

Location: `crates/satspath-core/src/bip353.rs:240-244`, `DohTxtResolver` **Impact:** Default `Strict` mode **always fails** because `DohTxtResolver` marks all records `dnssec_validated: false`.

```
// bip353.rs - DohTxtResolver CANNOT validate DNSSEC locally
#[async_trait]
impl DnsTxtResolver for DohTxtResolver {
    async fn query_txt(&self, fqdn: &str) -> Result<Vec<DnsTxtRecord>> {
        // ... fetch from cloudflare-dns.com ...
        dnssec_validated: false,  // ALWAYS false
    }
}
```

Result: `resolve_bip353("alice@domain.com")` → `Err(DnssecUnavailable)` on mainnet.

Mitigation Required: - Integrate a local DNSSEC validator (e.g., `trust-dns-resolver` with DNSSEC) - Or: document that BIP-353 requires external DNSSEC-validating resolver - Add `DnssecPolicy::DevInsecure` guard: require explicit CLI flag + env var

---

3. CRITICAL: Swap Engine — Refund/Claim Builders Not Implemented

Location: `crates/satspath-swaps/src/execution_gate.rs:4-18`, `submarine.rs:174-202` **Impact:** Submarine/Reverse/Chain swaps **cannot complete refund path** if Boltz fails to pay invoice. Funds locked in HTLC until CLTV expiry (hours/days).

```rust
// execution_gate.rs - ALL swap kinds return false
pub fn claim_refund_builders_available(kind: SwapKind) -> bool {
    match kind {
        SwapKind::Submarine | SwapKind::Reverse | SwapKind::Chain => false,
    }
}

// submarine.rs - Refund attempt returns error, doesn't build tx
async fn attempt_submarine_refund(...) -> Result<String> {
    Err(SwapError::Key("Refund tx building not yet implemented...".into()))
}
```

Risk: User deposits on-chain BTC → Boltz fails to route → **no automatic refund** → user must manually construct HTLC refund transaction using redeem script.

Mitigation Required: - Implement `tx_builder.rs` with `bitcoin` crate for HTLC/Taproot refund paths - Add integration tests with Boltz testnet for full lifecycle (deposit → timeout → refund) - Document manual recovery procedure in `README.md`

---

4. CRITICAL: Ark Ownership Proof — Challenge Message Binding Weakness

Location: `crates/satspath-core/src/ark.rs:190-197`, `verify_ark_ownership_proof:253-257` **Impact:** Challenge message prefix check allows message extension attacks.

```rust
// ark.rs - ark_ownership_challenge
pub fn ark_ownership_challenge(alias: &str, ark_server: &str, receiver_pubkey: &str, nonce: &str) -> St
    format!("satspath-proof:{alias}:ark:{ark_server}:{receiver_pubkey}:{nonce}")
}

// verify_ark_ownership_proof - WEAK CHECK
let expected_prefix = ark_ownership_challenge(alias, &pointer.server, &pointer.receiver_pubkey, "");
if !proof.message.starts_with(&expected_prefix) || proof.message == expected_prefix {
    return Err(SatsPathError::InvalidSignature);
}
```

Attack: Attacker gets valid signature for `nonce="legit"`, then presents `message = "satspath-proof:alice@...:ark:...:`
Prefix matches, signature verifies, but message semantics changed.

Fix: Require exact message match or structured parsing (e.g., require `nonce` at end with delimiter).

---

5. CRITICAL: Profile Substitution in ChainResolver (SEC-02) — Only Checks Canonical Form

Location: `crates/satspath-core/src/resolver.rs:48-61` **Impact:** Attacker registers `alíce@example.com` (with accented í) → canonicalized to different hash but displays identically to `alice@example.com`.

```
let requested_canonical = canonical_identifier(alias);  // "alice@example.com"
let returned_canonical = canonical_identifier(&profile.profile.alias);  // "alíce@example.com" → differ
```

Wait — `canonical_identifier` lowercases ASCII only. Non-ASCII passes through. This **is** a homograph attack vector.

Fix: Reject non-ASCII in alias during registration/resolution, or use IDNA2008/punycode normalization.

High Severity Findings

6. HIGH: On-chain Fee Threshold Logic Inconsistency

Location: `docs/protocol.md:169` vs `crates/satspath-router/src/router.rs:111` **Protocol Spec:** `hourFee  10 sat/vB` → On-chain **Router Code:** `fastest_fee  20 sat/vB` → On-chain

```
// router.rs - Uses fastestFee (next block), threshold 20
pub fn is_onchain_fee_acceptable(estimate: &FeeEstimate) -> bool {
    estimate.fastest_fee <= 20
}
```

Risk: Protocol promises "hour fee 10", but router uses "next-block fee 20". During fee spikes, user gets on-chain confirmation in hours (not <10 min) or pays 2x expected fee.

Fix: Align router to protocol spec (use `hour_fee` with threshold 10) OR update protocol doc to match implementation.

---

7. HIGH: Registry Downgrade Attack (SEC-03) — Only Checks `updated_at`

Location: `crates/satspath-core/src/registry.rs:66-75` **Impact:** Attacker with write access to registry file can replace profile with older version (removing Lightning method, forcing on-chain).

```
if let Some(existing) = self.data.profiles.get(&key) {
    if signed.profile.updated_at < existing.profile.updated_at {  // ONLY timestamp check
        return Err(...)
    }
}
```

Bypass: Attacker sets `updated_at` to future timestamp (e.g., `now + 1 year`). Registry accepts it. Profile now "newer" but stripped of methods.

Fix: - Require monotonic `updated_at` AND method set superset check - Or: version-pinned profiles with append-only registry (Nostr/DHT)

---

8. HIGH: No Replay Protection on Payment Profiles

Location: `crates/satspath-core/src/profile.rs:116-141`, `crypto.rs:33-47` **Impact:** Old signed profile (with old methods) can be replayed indefinitely until `expires_at`.

Scenario: User rotates keys, publishes new profile. Attacker intercepts old profile and serves it to payers. Payers route to old Lightning Address (attacker-controlled).

Fix: - Add `nonce` or `sequence` to profile, require strictly increasing - Short `expires_at` (e.g., 24h) with automatic refresh - Client-side pinning: remember `identity_pubkey` + `updated_at` per alias

---

9. HIGH: Invite Flow — No Sender Authentication

Location: `crates/satspath-core/src/lib.rs:66-84`, `docs/protocol.md:211-232` **Impact:** Anyone can create invite for any alias with any amount. Receiver sees invite but cannot verify sender.

```rust
pub fn create_invite(alias: &str, amount_sats: u64) -> Invite {
    let digest = Sha256::digest(privacy::canonical_identifier(alias).as_bytes());
    let alias_hash = hex::encode(digest);
    let claim_url = format!("https://satspath.local/claim?alias_hash={}&amount={}", &alias_hash[..16], a
    // No sender identity, no signature, no expiry enforcement on claim
}
```

Attack: Attacker spams invites to target alias with large amounts. Receiver claims, publishes profile. Attacker pays… but wait, invite has no binding to sender.

Fix: Sign invite with sender's identity key. Include `sender_pubkey`, `created_at`, `expires_at`. Receiver verifies sender owns claimed identity.

---

10. HIGH: Canonical JSON Serialization Not Truly Deterministic

Location: `crates/satspath-core/src/crypto.rs:23-31`, `docs/protocol.md:100-101` **Claim:** "Field order preserved via Rust's derived `Serialize`"

Reality: `serde_json::to_string` preserves struct field order **only for the same binary**. Different Rust versions, feature flags, or crate compilations **can change field order**. This breaks signature verification across versions.

```rust
pub fn canonical_profile_bytes(profile: &PaymentProfile) -> Result<Vec<u8>> {
    let json = serde_json::to_string(profile)  // NOT canonical!
        .map_err(|e| SatsPathError::SerializationError(e.to_string()))?;
    Ok(json.into_bytes())
}
```

Fix: Use true canonical serialization (e.g., `serde_cbor` with deterministic map ordering, or implement custom canonical JSON with sorted keys).

Medium Severity Findings

11. MEDIUM: Arkade Opaque URI — No Ownership Verification Possible

Location: `crates/satspath-core/src/ark.rs:80-95`, `validate_arkade_opaque_uri` **Impact:** `opaque_uri` (ark1q… or ark:…) accepts any string starting with prefix. No proof that user controls the Arkade address.

```rust
pub fn validate_arkade_opaque_uri(uri: &str) -> Result<()> {
    let lower = trimmed.to_ascii_lowercase();
    if !lower.starts_with("ark1") && !lower.starts_with("ark:") { ... }
    assert_no_private_material(trimmed)?;  // Only check
    Ok(())
}
```

Risk: User pastes attacker's `ark1q...` address. SatsPath routes to attacker's Arkade account. No cryptographic binding to identity.

Mitigation: Document clearly: `opaque_uri` = **preview only**, `execution: manual_wallet`. Never auto-execute.

---

12. MEDIUM: Fee Estimation Fallback is Dangerous Default

Location: `crates/satspath-router/src/fees.rs:14-24`

```rust
pub fn fallback() -> Self {
    FeeEstimate {
        fastest_fee: 10,  // 10 sat/vB = ~$1.50 for 141 vByte
        half_hour_fee: 7,
        hour_fee: 5,
        economy_fee: 3,
        minimum_fee: 1,
    }
}
```

Risk: API fails → router assumes 5 sat/vB (hour fee) → selects on-chain for large amounts. Actual fee may be 50 sat/vB → user pays 10x or transaction stalls.

Fix: On API failure, **fail closed** (return error) rather than optimistic fallback. Or require explicit `--assume-fee` flag.

---

13. MEDIUM: Lightning Dust Threshold Not Enforced in Router

Location: `crates/satspath-router/src/router.rs:75-99`

```rust
// Protocol: "The dust threshold must NOT block Lightning route selection."
if req.amount_sats < LIGHTNING_THRESHOLD_SATS {  // 100,000 sats
    if let Some(ln) = methods.iter().find(|m| is_lightning_available(m)) {
        return Ok(RouteQuote { ... });  // No dust check!
    }
}
```

Issue: Lightning invoices have minimum amounts (often 1 sat, but some LSPs enforce higher). Router selects Lightning for 1 sat without verifying the LNURL/Address accepts it.

Fix: Fetch LNURL metadata first to check `minSendable` before selecting Lightning.

---

14. MEDIUM: Privacy — Identifier Hash is SHA-256(email) — Trivially Reversible

Location: `crates/satspath-core/src/privacy.rs:7-11`, `peer_registry.rs:29-34`

```rust
pub fn identifier_hash(identifier: &str) -> String {
    let digest = Sha256::digest(canonical_identifier(identifier).as_bytes());
    hex::encode(digest)
}
```

Impact: Email addresses are low-entropy. Attacker with registry dump can rainbow-table all common emails.

Fix: Use HMAC with secret pepper, or Argon2id. Or accept this is "privacy hint only" and document clearly.

---

15. MEDIUM: BOLT12 Invoice Validation — Only Checks Prefix

Location: `crates/satspath-router/src/scoring.rs:249-251`, `validation.rs:229-231`

```
fn bolt11_like(value: Option<&str>) -> Option<&str> {
    value.filter(|invoice| invoice.starts_with("lnbc") || invoice.starts_with("lntb"))
}
```

Issue: Accepts any string starting with `lnbc`/`lntb`. No validation of invoice structure, amounts, expiry, or signature.

Fix: Use `bolt12` crate to parse and validate offer/invoice, or reject BOLT12 in preview mode.

Low Severity / Defense-in-Depth

16. LOW: Private Material Detection — String Matching Only

Location: `crates/satspath-core/src/validation.rs:79-161`

```
let blocked = ["xprv", "tprv", "seed phrase", "mnemonic", "private_key", ...];
if let Some(term) = blocked.iter().find(|term| lower.contains(**term)) { ... }
```

Bypass: `xprv` encoded as `xprv` with zero-width spaces, base64, or split across JSON fields.

Fix: This is a defense-in-depth layer (correct). Document it's not a cryptographic guarantee.

---

17. LOW: Swap Store — No Key Derivation Function Specified

Location: `crates/satspath-swaps/src/swap_store.rs:44-55`

```
pub fn open_encrypted(key: [u8; 32]) -> Result<Self>
```

Issue: Caller must provide 32-byte key. No standard KDF (PBKDF2/Argon2) recommended. Weak passwords → weak encryption.

Fix: Add `open_with_password(password: &str)` using Argon2id, or document required KDF.

---

18. LOW: Ark Route Planning — No Sender Balance Check

Location: `crates/satspath-router/src/ark_routes.rs:20-91` **Issue:** `plan_ark_route` assumes sender has funds on Ark/Lightning/on-chain. No validation.

Fix: Add balance check in swap engine before creating swap.

Black-Box Protocol Analysis

19. PROTOCOL: Routing Algorithm Edge Cases

Spec: `docs/protocol.md:156-176`

Scenario	Spec Behavior	Potential Issue
Amount = 99,999 sats, Lightning + On-chain	Lightning	OK
Amount = 100,000 sats, Lightning + On-chain (fee=5)	On-chain	Lightning skipped despite being cheaper
Amount = 500,000 sats, On-chain (fee=25) + Ark	Ark	Correct fallback
Amount = 500,000 sats, NO Lightning, NO On-chain, NO Ark	Error	Correct
Amount = 1 sat, Lightning Address exists	Lightning	Dust risk (see #13)

Finding: Threshold at exactly 100,000 sats creates discontinuity. At 99,999 → Lightning (instant, ~1 sat fee). At 100,000 → On-chain (10 min, ~700 sat fee @ 5 sat/vB). User incentivized to split payments.

---

20. PROTOCOL: Invite Flow — No Rate Limiting

Spec: `docs/protocol.md:211-232`

Attacker can generate unlimited invites for target alias, causing: - Spam in receiver's claim inbox - Registry pollution (if invites stored server-side) - Social engineering: "You received 10 BTC, claim now!"

Fix: Rate limit invite creation per sender fingerprint/IP. Add `expires_at` to invite (currently only in `InviteRecord`, not `Invite`).

---

21. PROTOCOL: BIP-353 — Consumer Email Domains Excluded

Spec: `docs/protocol.md:250-258`

> "Consumer email domains (e.g. gmail.com) cannot use BIP-353 — they fall back to platform verification / the invite flow."

Implication: 90%+ of users (Gmail, Outlook, Yahoo) **cannot use DNSSEC resolution**. They rely on centralized platform verification, defeating decentralization.

Fix: Document this limitation clearly. Explore Nostr NIP-05 as alternative for consumer domains.

---

22. PROTOCOL: Mainnet Preview — No Execution, But Invoice Fetching

Spec: `docs/protocol.md:178-205`

> "It must not: execute Lightning payments… but may display BOLT11 invoice string only when explicitly fetched by the caller"

Risk: Caller fetches BOLT11 invoice → displays to user → user pays. But invoice could be: - For different amount - Expired - From different node (if LNURL compromised)

Fix: Validate fetched invoice matches requested amount, hasn't expired, and (if possible) matches receiver pubkey in profile.

Positive Security Observations

Area	Strength
Signature Verification	Mandatory on ALL profile resolution paths (local + remote)
Private Material Rejection	Comprehensive blocklist + heuristic detection in validation
Ownership Proofs	Multi-tier (Cryptographic/DomainControl/SelfAsserted) with replay protection
Swap Storage	Encrypted-by-default (AES-256-GCM), refuses sensitive data without key
Registry	Downgrade protection (SEC-03), conflict detection on pubkey mismatch
Resolver	Profile substitution attack mitigation (SEC-02)
VTXO Verification	Full DAG reconstruction + on-chain anchoring verification (Ark Bridge)
Fee Fallback	Conservative (5 sat/vB hour fee) rather than optimistic
Test Coverage	Extensive unit + integration tests for crypto, ownership, routing, swaps

Recommended Remediation Priority

Priority	Findings	Effort
P0 (Before Mainnet)	#1 Fee API redundancy, #2 BIP-353 DNSSEC validator, #3 Swap refund builders, #4 Ark challenge binding, #5 Homograph attack	High
P1 (Before Testnet w/ Real Value)	#6 Fee threshold alignment, #7 Registry downgrade, #8 Profile replay, #9 Invite signing	Medium
P2 (Hardening)	#10 Canonical JSON, #11 Opaque URI docs, #12 Fee fallback fail-closed, #13 LN dust check, #14 Peppered hashes, #15 BOLT12 validation	Low-Medium

Testing Recommendations

White-Box (Add to CI)

1. **Property-based tests** for canonical serialization (proptest)
2. **Fuzzing** on profile parsing, signature verification, URI parsing
3. **Mutation testing** on validation logic (cargo-mutants)
4. **Cross-version signature compatibility** tests

Black-Box (Protocol Level)

1. **Network simulation** tests: MITM fee API, DNS hijacking, registry corruption
2. **Invite flow abuse** testing: rate limits, spam, social engineering
3. **Routing edge cases**: fee boundary oscillation, method removal during payment
4. **Ark VTXO verification** against malicious ASP (already well-tested in `blackboxSec.test.ts`)

Appendix: File Reference Map

Component	Files
Core Crypto	`crates/satspath-core/src/crypto.rs`
Profile/Validation	`crates/satspath-core/src/profile.rs`, `validation.rs`
Ownership Proofs	`crates/satspath-core/src/ownership.rs`
Registry/Resolver	`crates/satspath-core/src/registry.rs`, `resolver.rs`
BIP-353	`crates/satspath-core/src/bip353.rs`, `bip353_publish.rs`
Router/Fees	`crates/satspath-router/src/router.rs`, `fees.rs`, `scoring.rs`
Ark	`crates/satspath-core/src/ark.rs`, `crates/satspath-router/src/ark_routes.rs`
Swaps	`crates/satspath-swaps/src/` (submarine, reverse, chain, boltz_client)
Ark Bridge (TS)	`ark-bridge/src/vtxoDAGVerification.ts`, `arkdProvider.ts`, `blackboxSec.test.ts`
Docs	`docs/protocol.md`, `docs/threat_model.md`, `docs/architecture.md`

End of Audit Report

This report covers the prototype codebase as of commit `HEAD`. Findings reflect the "hackathon prototype" threat model — not production-ready software.